

UNIT 8

Software Reverse Engineering

8.1 Basics of software reverse engineering

Software reverse engineering is a process of recovering the design, requirement specifications, and functions of a product from an analysis of its code. It builds a program database and generates information from this. This article focuses on discussing reverse engineering in cyber forensic investigation.

8.1.1 Reverse software engineering to understand data

Reverse engineering of data occurs at different levels of abstraction. It is often the first reengineering task.

1. At the program level, internal program data structures must often be reverse-engineered as part of an overall reengineering effort.
2. At the system level, global data structures (e.g., files, databases) are often reengineered to accommodate new database management paradigms (e.g., the move from flat file to relational or object-oriented database systems).

Internal Data Structures

Reverse engineering techniques for internal program data focus on the definition of classes of objects.

1. This is accomplished by examining the program code with the intent of grouping related program variables.
2. In many cases, the data organisation within the code identifies abstract data types.
3. For example, record structures, files, lists, and other data structures often provide an initial indicator of classes.

8.1.2 Reverse software engineering to understand processing:

To understand, processing begins with an attempt to understand and then extract procedural abstractions represented by the source code. To understand procedural abstractions, the code is analysed at varying levels of abstraction: system, program, component, pattern, and statement.

1. Each of the programs that make up the application system represents a functional abstraction at a high level of detail. A block diagram, representing the interaction between these functional abstractions, is created.
2. Each component performs some subfunction and represents a defined procedural abstraction. A processing narrative for each component is developed.

For large systems, reverse engineering is generally accomplished using a semiautomated (partial automation) approach. Automated tools can be used to help you understand the semantics of existing code. The output of this process is then passed to restructuring and forward engineering tools to complete the reengineering process.

8.1.3 Steps of Software Reverse Engineering

1. **Collection information:** This step focuses on collecting all possible information (i.e., source design documents, etc.) about the software.
2. **Examining the information:** The information collected in step-1 is studied so as to get familiar with the system.
3. **Extracting the structure:** This step concerns identifying program structure in the form of a structure chart where each node corresponds to some routine.
4. **Recording the functionality:** During this step processing details of each module of the structure, charts are recorded using structured language like decision table, etc.
5. **Recording data flow:** From the information extracted in step-3 and step-4, a set of data flow diagrams is derived to show the flow of data among the processes.
6. **Recording control flow:** The high-level control structure of the software is recorded.
7. **Review extracted design:** The design document extracted is reviewed several times to ensure consistency and correctness. It also ensures that the design represents the program.
8. **Generate documentation:** Finally, in this step, the complete documentation, including SRS, design document, history, overview, etc., is recorded for future use.

8.1.4 Reverse engineering tools

Reverse engineering tools accept source code as input and produce a variety of structural, procedural, data, and behavioural designs. Reverse engineering, if done manually, would consume a lot of time and human labour and hence must be supported by automated tools. Some of the tools are given below:

1. CIAO and CIA: A graphical navigator for software and web repositories and a collection of Reverse Engineering tools.
2. Rigi: A visual software understanding tool.
3. Bunch: A software clustering/modularisation tool.
4. GEN++: An application generator to support the development of analysis tools for the C++ language.
5. PBS: Software Bookshelf tools for extracting and visualising the architecture of programs.

8.1.5 Reverse engineering techniques

1. *Static analysis:* Analysing software without executing it, such as decompiling code to a higher-level language.
2. *Dynamic analysis:* Analysing software while it's running, observing its behaviour and interactions with the operating system.
3. *Disassembly and debugging:* Breaking down software into its basic instructions and tracing their execution.

8.1.6 Challenges of reverse engineering

1. *Obfuscation and encryption*: Malware authors often use techniques to make their code harder to analyse.
2. *Code complexity*: Malware can be highly complex and difficult to understand, especially for beginners.
3. *Resource-intensive*: Reverse engineering can be time-consuming and require specialised knowledge and tools.

8.2 Generic roles of software reverse engineering

Generally, software engineering plays a significant role in several aspects of the software industry. Some of these roles are explained below.

8.2.1 Defending Against Software Targets for Viruses, Worms, and Other Malware

Malicious software (malware) is often crafted to exploit software vulnerabilities. Reverse engineering the WannaCry ransomware allowed security experts to develop a “kill switch” by finding and registering a specific domain name embedded in its code. Reverse Engineering helps by:

1. Analysing malware samples: Using tools like IDA Pro, Ghidra, or Radare2 to dissect and understand malware behaviour.
2. Identifying entry points: Studying how viruses infect software (e.g., code injection, hooking functions).
3. Developing countermeasures: Patching vulnerabilities and designing antivirus signatures.
4. Understanding malware propagation: Dissecting worms to understand their replication techniques.

8.2.2 Improving Third-Party Software Libraries through reverse engineering

Organisations often rely on third-party libraries without access to source code, which might be outdated, insecure, buggy, or missing documentation. However, the best practice tip is to always respect licenses! Reverse engineering third-party libraries might violate terms of service unless for interoperability, security research, or with explicit permission. Reverse engineering allows for:

1. Security Audits: Analysing libraries for hidden backdoors or insecure functions.
2. Performance Optimisation: Identifying inefficient code or unnecessary dependencies.
3. Bug Fixes: Understanding behaviour to patch errors locally when the vendor is slow to update.

8.2.3 Identifying hostile code: Buffer overflow and unexpected inputs

Buffer overflow

A buffer overflow occurs when data overflows from one memory buffer into another, potentially allowing attackers to execute arbitrary code. Reverse engineering techniques to identify buffer overflow are:

1. Static Analysis: Reviewing binary for dangerous functions (e.g., strcpy, sprintf, gets).
2. Dynamic Analysis: Fuzzing the application to force buffer overflows.
3. Symbolic Execution: Analysing all possible inputs and paths to find unsafe conditions.

Typical indicators of buffer overflow include a lack of boundary checking, use of unsafe memory functions, and abnormal crash behaviour. The commonly used defence mechanisms are stack canaries, address space layout randomisation (ASLR), and safe programming libraries.

Provision of unexpected inputs

Many attacks succeed because software fails to properly handle unexpected inputs (e.g., special characters, very large inputs, malformed data). Common threats known with unexpected inputs are SQL injection, cross-site scripting (XSS) and format string vulnerabilities. Thus, reverse engineering focus areas include input parsing functions, error handling routines, and input validation logic.

8.3 Relationship between software reverse engineering and cyber forensics

Software reverse engineering is a crucial technique in forensic investigations, particularly when analysing malicious software (malware) or understanding how an attack was carried out. It involves examining software without access to the source code to deduce its functionality, algorithms, and internal workings. This allows investigators to understand how attackers exploited vulnerabilities, identify the payload of malicious software, and trace the origin of attacks. Software reverse engineering and cyber forensics are related in the following ways:

8.3.1 Understanding evidence

Digital Forensics: In cybercrime investigations, reverse engineering is used to analyse malware or software to understand how it functions, identify its source, and determine its impact. This can help trace back to the perpetrator or reveal their methods.

Physical Evidence: In cases involving hardware (e.g., modified firearms or tampered devices), reverse engineering can help forensic experts understand how these items were altered or used in a crime.

8.3.2 Reconstruction of events

Reverse engineering can help recreate the sequence of events in a crime by analysing the tools or systems involved. For instance, examining a hacked computer system can reveal how an attack was executed, helping to piece together the timeline and methods used by the criminal.

8.3.3 Product analysis

In cases involving counterfeit products or intellectual property theft, reverse engineering can determine the authenticity of items. Forensic experts can compare the original design or functionality with the counterfeit to identify violations.

8.3.4 Developing countermeasures

By understanding how a crime was committed (e.g., through reverse engineering a hacking tool), law enforcement can develop better security measures or countermeasures to prevent future incidents.

8.3.5 Legal and ethical Implications

The application of reverse engineering in forensics often raises legal and ethical questions, particularly concerning privacy and intellectual property rights. Forensic experts must navigate these issues while ensuring that their methods adhere to legal.

8.4 Key aspects of reverse engineering in cyber forensics

(1) Malware analysis: Examining malicious software like viruses, worms, and trojans to understand their purpose, behaviour, and how they infect systems.

(2) Vulnerability discovery: Identifying weaknesses in software that attackers could exploit.

(3) Understanding attack techniques: Analysing how attackers leverage software to compromise systems and steal data.

(3) Developing countermeasures: Creating defences against specific threats and attack vectors.

(4) Why is reverse engineering important in cyber forensics

- a) *Incident investigation:* Determining the root cause of a security breach and the attacker's methods.
- b) *Threat intelligence:* Gaining insights into new or evolving threats, allowing for better preparedness.
- c) *Security enhancements:* Identifying and patching vulnerabilities in systems and applications.
- d) *Evidence collection:* Providing forensic evidence to legal authorities.

Reading Text

<https://www.geeksforgeeks.org/software-engineering-reverse-engineering/>

<https://www.quora.com/How-does-reverse-engineering-relate-to-forensics-and-solving-crimes>